

Unit: 6 Application Hardware Components and Deployment

Deploying a mobile application to the Google Play Store involves several steps to ensure that your app is ready for release and adheres to the Play Store guidelines. Here's a comprehensive guide:

Step 1: Prepare Your Application

1. Build a Release-Ready APK/AAB:

- Generate a **signed APK (Android Package)** or **AAB (Android App Bundle)** using Android Studio.
- Ensure the app is optimized, debugged, and ready for production.

Steps to Sign an APK/AAB:

- In Android Studio, go to **Build > Generate Signed Bundle / APK**.
- Create or use an existing **Keystore** to sign the app.
- Choose release mode and finish the process.

2. Optimize Your Application:

- Minimize the app size using **ProGuard** or **R8**.
- Test thoroughly on various devices and versions of Android.

3. Prepare a Privacy Policy:

- If your app collects user data, provide a clear and concise **Privacy Policy** URL.

Step 2: Register for a Google Play Developer Account

1. Sign Up:

- Go to the [Google Play Console](https://play.google.com/console).
- Use a Google account to register.

2. Pay the Registration Fee:

- Pay the one-time registration fee of \$25 (USD).

3. Complete the Account Setup:

- Fill out all required details, including organization or personal information.

Step 3: Set Up the Application in the Play Console

1. Create a New Application:

- Log in to the Google Play Console.
- Click **Create App** and provide:
 - App Name
 - Default language
 - App Type (App/Game)
 - Free or Paid

2. Provide App Details:

- Fill in the required metadata, including:
 - **Title:** The name of your app.
 - **Short Description:** A brief summary visible in the Play Store.
 - **Full Description:** Detailed information about the app.
- Add **screenshots** (for phones, tablets, etc.), a **feature graphic**, and an optional promo video.

3. Categorize Your App:

- Select the appropriate **category** and **tags** for your app.
- Answer the **content rating questionnaire** to classify your app.

4. Add Contact Information:

- Provide a support email and optional phone number and website.

Step 4: Set Up App Policies

1. Privacy Policy:

- Add the URL to your privacy policy in the designated field.

2. Content Guidelines:

- Ensure the app complies with Google Play's [Developer Policy](#).

3. Target Audience and Content:

- Specify the intended age group for your app.
- Disclose if the app includes ads.

Step 5: Upload the App Bundle or APK

1. Go to the “Release” Section:

- Navigate to **Release > Production > Create Release**.

2. Upload Your App:

- Drag and drop your signed APK or AAB file.
- Ensure the uploaded file matches the version code in the build.gradle file.

3. Add Release Notes:

- Provide details about the current release (e.g., new features, bug fixes).

Step 6: Finalize and Review

1. Run Pre-Launch Tests:

- Use Google Play’s automated pre-launch testing tools to detect potential issues.

2. Review and Fix Warnings:

- Address any warnings or errors flagged in the Play Console.

3. Set Pricing and Distribution:

- Choose whether the app is **Free or Paid**.
- Select the countries or regions where the app will be available.

4. Set Up Monetization (Optional):

- If applicable, configure **in-app purchases** or **subscriptions**.

Step 7: Publish the Application

1. Submit for Review:

- Click **Review Release** to confirm that all required sections are complete.
- Click **Start Rollout to Production** to publish your app.

2. Wait for Approval:

- Google typically reviews apps within 1–7 days (longer during holidays).
- You'll receive an email notification when your app is live.

Post-Publication Steps

1. Monitor Performance:

- Use the Play Console to track app performance, downloads, and user feedback.

2. Respond to Reviews:

- Engage with users by responding to reviews to improve ratings.

3. Release Updates:

- For new features or bug fixes, upload updated APKs/AABs following a similar process.

4. Promote Your App:

- Use marketing strategies and ads to reach a broader audience.

By following these steps, you can successfully deploy and manage your mobile application on the Google Play Store.

Fingerprint Authentication

Steps to Implement Fingerprint Authentication

1. Set Up Dependencies

- Use Android's **BiometricPrompt API** (recommended for Android 6.0 and above) for fingerprint authentication.
- Add necessary permissions in the AndroidManifest.xml:
- `<uses-permission android:name="android.permission.USE_BIOMETRIC" />`

2. Check Device Compatibility

- Verify if the device supports fingerprint authentication and if biometric data is enrolled:
- `val biometricManager = BiometricManager.from(context)`
- `if (biometricManager.canAuthenticate() ==`
`BiometricManager.BIOMETRIC_SUCCESS) {`
- `// Proceed`
- `}`

3. Create the BiometricPrompt Instance

What is BiometricPrompt?

BiometricPrompt is a built-in Android class that:

- Shows the fingerprint/face unlock pop-up.
 - Waits for the user to authenticate.
 - Gives you the result (success or failure).

Why do we use Executor?

An **Executor** tells Android on which thread the authentication result should be returned.

Example:

Often we use

```
executor = ContextCompat.getMainExecutor(this)
```

because UI updates must run on the **main thread**.

What is AuthenticationCallback?

This is an object that contains functions that are automatically called when authentication happens.

It has important override methods:

1. onAuthenticationSucceeded()

This function is called when the user is successfully authenticated.

Example use:

```
override fun onAuthenticationSucceeded(result: BiometricPrompt.AuthenticationResult) {  
    // Authentication success  
    Toast.makeText(activity, "Authentication Success!", Toast.LENGTH_SHORT).show()  
}
```

2. onAuthenticationFailed()

This function is called when the biometric input does **not match**.

Example use:

```
override fun onAuthenticationFailed() {  
    // Handle failure  
    Toast.makeText(activity, "Authentication Failed!", Toast.LENGTH_SHORT).show()  
}
```

Simplified Version of the Code

```
val biometricPrompt = BiometricPrompt(  
    activity,  
    executor,  
    object : BiometricPrompt.AuthenticationCallback() {  
  
        override fun onAuthenticationSucceeded(result: BiometricPrompt.AuthenticationResult) {  
            // Called when fingerprint/face is correct  
        }  
  
        override fun onAuthenticationFailed() {  
            // Called when fingerprint/face is NOT recognized  
        }  
    }  
)
```

)

CONCLUSION:

- **BiometricPrompt** → shows the fingerprint/face popup
- **Executor** → decides where the callback results will run
- **AuthenticationCallback** → has two important methods
 - **onAuthenticationSucceeded()** → fingerprint/face matched
 - **onAuthenticationFailed()** → fingerprint/face did not match

4. Configure Authentication

Before showing the fingerprint/face authentication dialog, you must **configure how the dialog will look**.

For this, Android gives a class called **BiometricPrompt.PromptInfo**.

This object decides:

- Title of the biometric popup
- Subtitle text
- What the Cancel button will show

You create this using a **Builder** pattern.

What is PromptInfo?

`BiometricPrompt.PromptInfo` is used to customize the biometric dialog that appears on the screen.

It controls what the user sees when the fingerprint popup opens.

What the Code Means

Code:

```
val promptInfo = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Authenticate")
    .setSubtitle("Use your fingerprint to continue")
    .setNegativeButtonText("Cancel")
    .build()
```

1. `.setTitle("Authenticate")`

This sets the **main heading** of the fingerprint dialog.

Example text visible to user:

Authenticate

2. `.setSubtitle("Use your fingerprint to continue")`

This is the **subtitle**, shown below the title.

It tells the user *what to do*:

"Use your fingerprint to continue"

3. `.setNegativeButtonText("Cancel")`

This sets the **Cancel** button on the popup.

When the user taps Cancel, authentication is stopped.

4. `.build()`

This creates the final **PromptInfo** object from the Builder.

CONCLUSION:

- **PromptInfo** = Design of the fingerprint popup
- **Title** = Main heading
- **Subtitle** = Instruction
- **Negative button** = Cancel button
- **build()** = Prepare the dialog

5. Trigger the Authentication

- Start the authentication process:
- `biometricPrompt.authenticate(promptInfo)`

Security Considerations

1. Data Protection

- Fingerprint data is stored in a secure hardware module (Trusted Execution Environment or TEE). Apps only receive a success/failure signal, not raw data.

2. API Usage

- Use only Android's BiometricPrompt API to ensure secure and standardized implementation.

3. Fallback Mechanism

- Provide alternative authentication methods (e.g., PIN or password) in case biometrics are unavailable.

4. Secure Communication

- Encrypt sensitive data and use secure network protocols (HTTPS) for transmitting authentication results.

5. Compliance with Policies

- Follow data privacy laws (like GDPR) and Google Play Store policies when handling user authentication.

This approach ensures robust fingerprint authentication with minimal security risks.

Interaction with Phone Calls

Interacting with Phone Calls in Mobile Applications

Mobile applications can interact with incoming and outgoing calls using telephony APIs provided by Android, such as the `TelecomManager`, `TelephonyManager`, and `PhoneStateListener`. However, strict user privacy and security guidelines must be followed.

Steps to Interact with Calls

1. Declare Necessary Permissions

- Include permissions in the `AndroidManifest.xml` file:
- `<uses-permission android:name="android.permission.READ_PHONE_STATE" />`
- `<uses-permission android:name="android.permission.READ_CALL_LOG" />`
- `<uses-permission android:name="android.permission.ANSWER_PHONE_CALLS" />`

2. Request Runtime Permissions

- Starting from Android 6.0 (API 23), request permissions at runtime for user consent.

3. Use Telephony APIs

Implement a `PhoneStateListener` to monitor call states:

```
val telephonyManager = context.getSystemService(Context.TELEPHONY_SERVICE) as
TelephonyManager
val phoneStateListener = object : PhoneStateListener() {
    override fun onCallStateChanged(state: Int, phoneNumber: String?) {
        when (state) {
            TelephonyManager.CALL_STATE_RINGING -> {
                // Handle incoming call
            }
            TelephonyManager.CALL_STATE_OFFHOOK -> {
                // Handle ongoing call
            }
            TelephonyManager.CALL_STATE_IDLE -> {
                // Handle call ended
            }
        }
    }
}
```

```
telephonyManager.listen(phoneStateListener,  
PhoneStateListener.LISTEN_CALL_STATE)
```

4. Ensure Privacy and Security

- Do not collect sensitive call details (e.g., phone numbers) without explicit user consent.
- Avoid storing call logs unless absolutely necessary and secure any stored data.

Example Use Case:

Spam Call Detection App

- The app monitors incoming calls using PhoneStateListener.
- Compares the phone number with a database of spam numbers.
- If a match is found, it alerts the user with a notification:
"This is a suspected spam call."

Privacy and Security Measures

1. **Permission Transparency:** Clearly explain the purpose of permissions to the user.
2. **Data Minimization:** Access only essential call-related data.
3. **User Control:** Allow users to opt-out of features that interact with calls.
4. **Policy Compliance:** Adhere to platform policies (e.g., Google Play) and privacy laws like GDPR.

By following these practices, apps can safely interact with phone calls while respecting user privacy.

Key UI Design Principles for Android TV Applications

1. **Simplicity and Clarity**

- Use minimal and essential UI elements to avoid clutter on large screens.
- Ensure text and visuals are clear and legible from a distance.

2. **Remote Control Navigation**

- Design for D-pad (directional pad) input with focusable elements.
- Provide clear visual cues for focused items.

3. **Large Screen Optimization**

- Use large fonts and scalable layouts to accommodate various TV resolutions.
- Ensure a content-first design with high-quality visuals.

4. **Seamless Navigation and Performance**

- Ensure smooth navigation with consistent global and back navigation behaviours.
- Optimize performance by using lazy loading and smooth animations.

5. **Accessibility and Discovery**

- Support voice input with Google Assistant and ensure high contrast for readability.
- Use the Recommendations API to surface content on the Android TV home screen.

By adhering to these principles, developers can create user-friendly and visually appealing TV applications.

Use Cases for Android TV Applications

1. Streaming Media Apps (e.g., Netflix, YouTube)

- Android TV is primarily used for streaming video content such as movies, TV shows, and live events. Apps like Netflix, YouTube, and Disney+ leverage the large screen and high-quality video playback features of Android TV.
- **Leverage of Hardware:** Utilizes high-definition resolution, large screen size, and HDMI support for immersive media consumption. Android TV's remote control functionality and voice search features (via Google Assistant) make browsing content easy and intuitive.

2. Gaming Apps (e.g., Android TV Games, Stadia)

- Android TV supports a variety of games, both casual and high-quality. Games can be played directly on the TV using a remote or Bluetooth controllers.
- **Leverage of Hardware:** The large screen provides an excellent platform for gaming, while the Android TV hardware supports high-performance gaming with controllers and cloud gaming services like Google Stadia. It also offers low-latency gaming experiences.

3. Live TV and DVR Apps (e.g., Sling TV, Hulu Live)

- Apps that provide live TV streaming and DVR functionality, enabling users to watch live broadcasts, record programs, and view on-demand content.
- **Leverage of Hardware:** Android TV supports seamless integration with set-top boxes and cable hardware, facilitating live TV streaming with high-quality video output. The built-in voice search allows users to easily find and manage live TV programs.

4. Home Automation and Smart Home Control Apps (e.g., Google Home, SmartThings)

- Android TV can serve as a hub for controlling smart home devices, such as lights, thermostats, security cameras, and other IoT devices.
- **Leverage of Hardware:** The Android TV remote or voice commands (via Google Assistant) can be used to control connected smart devices. Integration with Android

TV's Android Things platform allows seamless operation and monitoring of smart devices.

5. Fitness and Health Apps (e.g., Google Fit, Peloton)

- These apps allow users to follow workout routines, monitor health data, or stream fitness content directly to the TV.
- **Leverage of Hardware:** Android TV's large screen is ideal for viewing workout videos and tracking fitness goals. Some apps leverage additional features like Chromecast to cast workouts from mobile devices to the TV.

How These Use Cases Leverage Unique Features of Android TV Hardware:

- **Large Screen Display:** Most use cases, such as media streaming, gaming, and fitness, take advantage of the large screen for better viewing, providing an immersive experience.
- **Voice Interaction:** Google Assistant on Android TV allows for hands-free control of apps (e.g., searching content, controlling smart home devices) which enhances user experience.
- **Seamless Connectivity:** Android TV offers connectivity options (e.g., Bluetooth, HDMI) for external devices like gaming controllers, sound systems, and smart home accessories, allowing for versatile app use cases.
- **HDMI and High-Definition Support:** Perfect for video streaming, live TV apps, and gaming, where high resolution and smooth performance are critical.

These unique features make Android TV a versatile platform for various use cases, providing enhanced interactivity and high-quality multimedia experiences.

Role of Wearable Device Faces in Wearable Applications

A **wearable device face** (also known as a watch face) is the primary visual interface that users interact with on a wearable device, such as a smartwatch. It serves as the central hub for displaying essential information like the time, date, and other personalized data, making it the first thing users see when they check their device.

The design and functionality of the watch face are critical because it directly impacts the user experience. A well-designed watch face provides the right balance between aesthetics, usability, and the display of relevant data. Users can customize the appearance of their watch faces based on their preferences, ensuring that the information displayed is both useful and engaging.

Role of Face Complications in Enhancing User Experience

Complications are small, interactive data points or widgets that can be added to the watch face, offering additional information beyond the time. These can include details like weather updates, step count, battery life, heart rate, upcoming calendar events, or notifications. By adding complications to a watch face, developers allow users to access vital information quickly, without needing to navigate through menus or apps.

Benefits of Face Complications:

1. **Increased Functionality:** Complications provide real-time, glanceable information directly on the watch face, saving users time and effort by eliminating the need to open specific apps.
2. **Personalization:** Users can customize their watch faces by selecting complications that are most relevant to them (e.g., fitness data, news, or weather), tailoring their device experience to fit their lifestyle.
3. **Efficiency:** Complications allow users to monitor important data in a non-intrusive way, making it easy to track things like physical activity, battery status, and appointments at a glance, enhancing the overall usability of the device.
4. **Context Awareness:** Complications can be context-sensitive, adapting the information displayed based on the user's activity, location, or preferences. For instance, fitness complications may display heart rate or steps during a workout and shift to weather information later in the day.

In wearable applications, **watch faces** serve as the primary point of interaction, while **complications** enhance the user experience by displaying useful, personalized data on the watch face. By offering immediate access to important information, complications optimize the watch's functionality, improve usability, and ensure a more engaging and efficient experience for the user.

Process of Creating and Customizing a Watch Face for Wearable Devices

Creating and customizing a watch face for wearable devices (such as Android Wear or Apple Watch) involves several key steps, from design to implementation. Below is a high-level overview of the process:

1. Design the Watch Face Layout

- **Visual Design:** Start by creating a visual design for the watch face. This includes choosing the style (analog or digital), determining the color scheme, and selecting the fonts and icons. Keep in mind that the design must be visually appealing yet functional, considering the limited screen space.
- **Consider User Preferences:** Plan for customization options like changing colors, layouts, or displaying different complications (e.g., weather, heart rate, steps, etc.). Offering variety will improve user engagement.

2. Develop the Watch Face

- **Use the Watch Face APIs:** For Android, developers can use the **Watch Face API** provided by Google. For Apple Watch, **WatchKit** is used for building watchOS applications. These APIs allow developers to access and modify watch face components, add complications, and manage screen rendering.
- **Add Components:** Key components like **time**, **date**, **complications**, and **hands** (in analog faces) need to be programmed. Use the appropriate classes for rendering time and other data points:
 - For Android, use Canvas for custom drawing.
 - For Apple Watch, use WKInterfaceObject for building the watch face elements.
- **Complications:** Add complications to show additional data like step count, battery life, heart rate, or other relevant metrics. For Android, use ComplicationData and ComplicationProvider to link complications to the watch face.

3. Test and Debug the Watch Face

- **Test on Real Devices:** Test the watch face on real devices or simulators/emulators for various screen sizes, orientations, and resolutions.
- **Check Responsiveness:** Ensure that the watch face responds well to touch interactions (e.g., tapping complications or changing watch faces).
- **Test Battery Efficiency:** Optimize for battery consumption by using minimal power, reducing screen updates, and leveraging ambient modes when appropriate.

4. Customize User Preferences

- **Interactive Features:** Implement user controls, such as:
 - **Tap Gestures:** Allow users to tap on elements to open apps or show more detailed data.
 - **Swiping:** Enable users to swipe between different watch face styles or switch between different complications.
- **Personalization Options:** Allow users to customize aspects of the watch face, such as changing the color scheme, adding/removing complications, and selecting a layout.

Best Practices for Optimizing Performance and User Engagement

1. Battery Efficiency

- **Optimize Updates:** Limit how frequently the watch face updates, especially for components like time and complications. Consider using **ambient mode** (in Android) or **always-on display mode** to reduce power consumption when the watch face is not actively in use.
- **Efficient Rendering:** Avoid excessive animations or complex visual effects that might drain the battery. Use efficient rendering techniques, such as drawing only the elements that have changed.

2. Simplicity and Clarity

- **Focus on Key Information:** Ensure that the watch face is easy to read and glanceable. Avoid clutter and prioritize the most relevant data (time, date, fitness stats, etc.).
- **Legible Fonts and Icons:** Use large fonts for critical information, ensuring legibility from a distance. Keep icons simple and intuitive.

3. Optimize for User Interaction

- **Customizable Elements:** Provide users with the ability to customize the watch face to meet their preferences, such as choosing complications or changing the appearance (colors, background).
- **Seamless Navigation:** Make it easy for users to interact with the watch face, whether it's through tapping complications, swiping between faces, or accessing settings.

4. Contextual Relevance

- **Adaptive Complications:** Use context-aware complications that adapt to the user's activity. For example, display fitness data while the user is exercising, and switch to weather info later in the day.
- **Personalization:** Let users select the type of complications they want to see based on their daily needs (e.g., calendar events for business users, fitness data for active users).

5. Testing and Quality Assurance

- **Test Across Devices:** Ensure that the watch face performs well across various wearable devices with different screen sizes and resolutions.
- **Monitor Performance:** Regularly monitor the watch face's performance, particularly its impact on battery life and responsiveness.

Creating and customizing a watch face involves designing an appealing and functional layout, adding interactive and personalized elements, and ensuring the app performs efficiently. By optimizing for performance, keeping designs simple and intuitive, and offering customization

options, developers can significantly enhance user engagement and satisfaction. Proper testing and context-aware features further contribute to a seamless and enjoyable experience for users.

Five Key Differences Between Designing User Interfaces for Wearables and Traditional Mobile Devices

1. Screen Size and Resolution

- **Wearables:** Have much smaller screens compared to mobile devices. The limited screen size (typically 1.3 to 2 inches) means that the UI must be highly simplified, with fewer elements displayed at a time.
- **Mobile Devices:** Larger screens (typically 5 to 7 inches) allow for more complex layouts, more content on the screen, and the ability to display more interactive elements like buttons, icons, and text.

➤ **Impact on User Interaction:**

- **Wearables:** Due to the small screen, information must be presented in a concise, glanceable manner. Users interact with the device quickly, often with only a few taps or swipes.
- **Mobile Devices:** Users can engage in more detailed and prolonged interactions, including typing, scrolling, and using multiple apps at once, which requires more complex navigation.

2. Input Methods

- **Wearables:** Interaction is mostly done via touch gestures, physical buttons, or voice commands (e.g., using Google Assistant or Siri). Some devices support additional input like a rotating crown (on the Apple Watch) or wrist gestures.
- **Mobile Devices:** In addition to touch gestures, mobile devices support keyboards, larger touch areas, multi-touch gestures, and external inputs (e.g., mouse or stylus).

➤ **Impact on User Interaction:**

- **Wearables:** Input needs to be quick and simple. Actions like swiping, tapping, or rotating the crown should be intuitive and require minimal user effort. Complex interactions are avoided to cater to short, on-the-go usage.

- **Mobile Devices:** Can support more varied and detailed interactions. The larger screen and multiple input options allow for longer interactions, complex navigation, and multitasking.

3. Context of Use

- **Wearables:** Often used in motion, while walking, running, or during physical activities. They need to be highly functional in real-time, with a focus on glanceable information and quick access to essential data.
- **Mobile Devices:** Typically used in stationary or less active environments, allowing for more focused and detailed interactions. Users tend to engage with their phones for longer periods.
 - **Impact on User Interaction:**
- **Wearables:** The UI needs to be designed for quick glances and to provide immediate, actionable information. Features like voice control and haptic feedback are important for quick interaction without needing to look at the device.
- **Mobile Devices:** Users can focus for longer periods on mobile devices, allowing for detailed browsing, messaging, or app usage that requires complex layouts and longer interaction times.

4. Battery Life and Power Consumption

- **Wearables:** Have limited battery capacity (often lasting 1 to 2 days), which means energy-efficient UI design is crucial. The UI must minimize constant updates and power-intensive processes.
- **Mobile Devices:** Have larger batteries, allowing for more flexibility in UI design. However, battery management is still important, especially for power-hungry apps (e.g., gaming or video streaming).
 - **Impact on User Interaction:**

- **Wearables:** UI design must consider power efficiency, often using ambient or always-on display modes to conserve battery when the screen is not actively in use. This limits the use of heavy animations or frequent screen updates.
- **Mobile Devices:** While battery life is still a concern, users can generally afford to use more power-intensive features like video playback, animations, and more frequent updates without as much concern.

5. User Attention and Interaction Duration

- **Wearables:** Users generally engage with wearables for short periods. The interaction is often quick, such as checking the time, tracking fitness data, or receiving notifications.
 - **Mobile Devices:** Mobile devices tend to be used for longer, more in-depth interactions such as texting, browsing, gaming, or watching videos.
- **Impact on User Interaction:**
- **Wearables:** UI needs to be designed for brief and efficient interaction. The focus is on providing immediate access to the most important data, with minimal input required from the user.
 - **Mobile Devices:** UIs can afford to support prolonged interactions with more complex layouts, detailed text, and rich media. This allows for a more diverse range of activities, from work-related tasks to entertainment.

These differences highlight how the form factor, context of use, and user interaction needs vary significantly between wearables and mobile devices. Wearables require designs that are minimal, efficient, and optimized for quick, glanceable interactions, whereas mobile devices allow for more intricate and extended user engagement. By understanding these distinctions, designers can create interfaces that align with the capabilities and limitations of each device, ensuring a seamless user experience.